



# Développer avec **Flutter**

Le framework cross-platform by Google



# Hello, moi c'est **Florian**



Développeur web/mobile à OD&B

Développeur et photographe freelance

Formation en développement Full-Stack





# Pourquoi Flutter ?

Parce que





# 230 000 000 000

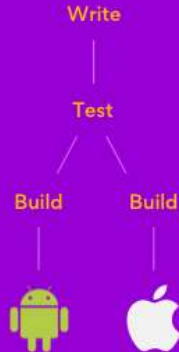
Nombre de téléchargements d'applications  
mobiles en 2021 (d'après App Annie)

(il y a aussi PWA mais bon...)

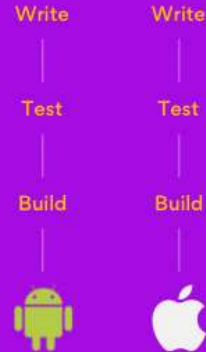


# Pourquoi de l'hybride ?

## HYBRID APPS



## NATIVE APPS



# Et si on ne se limitait **pas juste au mobile** ?



**Allez tester** les exemples web : <https://flutter.github.io/samples>

# Flutter répond **A un besoin**

- Un besoin de développer de plus en plus d'applications mobiles (ou autre).
- Un besoin de gagner en rapidité et un coût.
- Un besoin de ressentir du natif sous les doigts ou sous la souris.



Flutter Create

The logo features the text "Flutter Create" in a white, sans-serif font, centered over a dark blue background. The background is a grid of 40 small, colorful smartphone screens, each displaying a different Flutter app interface. The screens are arranged in 4 rows and 10 columns. The colors of the screens vary, including shades of blue, green, yellow, red, and black. The text "Flutter Create" is positioned in the middle of the grid, spanning across several screens.



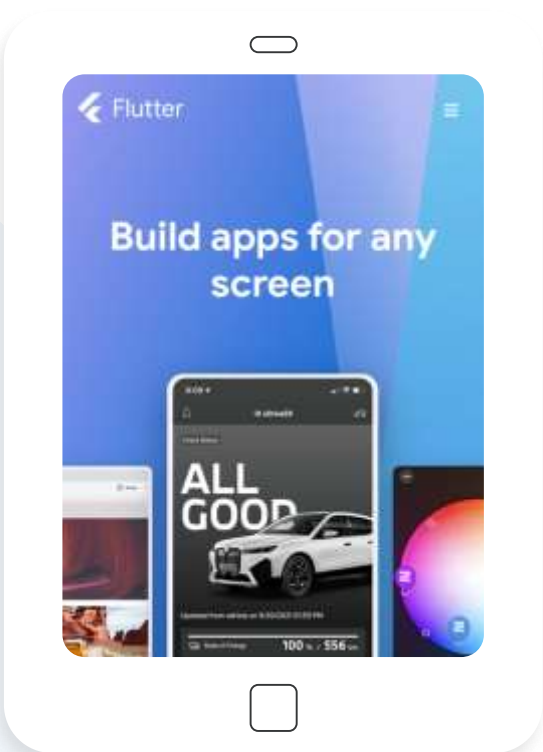




# C'est quoi **Flutter** ?

Une technologie super cool





## Le rendu avec Flutter

- Dessine le rendu de l'application via Skia (maintenant Impeller)
- Si des éléments changent, ils sont redessinés
- Un rechargement à chaud



# Le langage **Dart**

“Un langage optimisé pour l’UI d’applications rapides sur n'importe quelle plate-forme”



**Créé pour  
combler les  
lacunes du JS**

**Proche du TS et  
du Java**



## **Typé**

Fournir **un type** a chaque variable

## **Null safety**

Contrôler la **valeur null** dans une variable

## **Orienté objet**

Création **d’objets** avec des **classes**, polymorphisme...

# Une multitude de **plugins et packages**



Comparable à npm pour JS, Flutter dispose d'une multitude de packages disponibles sur [pub.dev](https://pub.dev). Il existe 3 types de librairies :



- Des packages développés exclusivement en Dart (Http, Bloc, Crypto...)
- Des packages développés exclusivement en Flutter
- Des plugins développés avec une partie native et avec un pont vers Flutter



Cette diversité de packages permet de ne pas re-développer certaines fonctionnalités et de faciliter l'application de certains concepts.

# Liste de **quelques plugins et packages** qui pourraient vous donner **des idées**

|          |                                           |
|----------|-------------------------------------------|
| <b>1</b> | <u><a href="#">Image picker</a></u>       |
| <b>2</b> | <u><a href="#">Shared preferences</a></u> |
| <b>3</b> | <u><a href="#">Geolocator</a></u>         |
| <b>4</b> | <u><a href="#">Lottie</a></u>             |

|          |                                   |
|----------|-----------------------------------|
| <b>5</b> | <u><a href="#">Fl chart</a></u>   |
| <b>6</b> | <u><a href="#">Share plus</a></u> |
| <b>7</b> | <u><a href="#">Camera</a></u>     |
| <b>8</b> | <u><a href="#">Flame</a></u>      |



# Comment l'utiliser ?

Comme ça





# Gérer **plusieurs versions** de Flutter

A la façon de NVM pour Node, vous pourrez retrouver [FVM pour Flutter](#)



Possibilité de gérer et de mettre en cache plusieurs versions du SDK Flutter.



Possibilité de gérer une version de Flutter spécifique pour chaque projet



# Les IDE pour Flutter



## Android Studio

Assez **lourd** mais **complet**  
et compilation android par  
la même occasion



## Visual Studio Code

Très **léger** et bonne prise en  
charge de Flutter grâce aux  
**plugins** VS Code

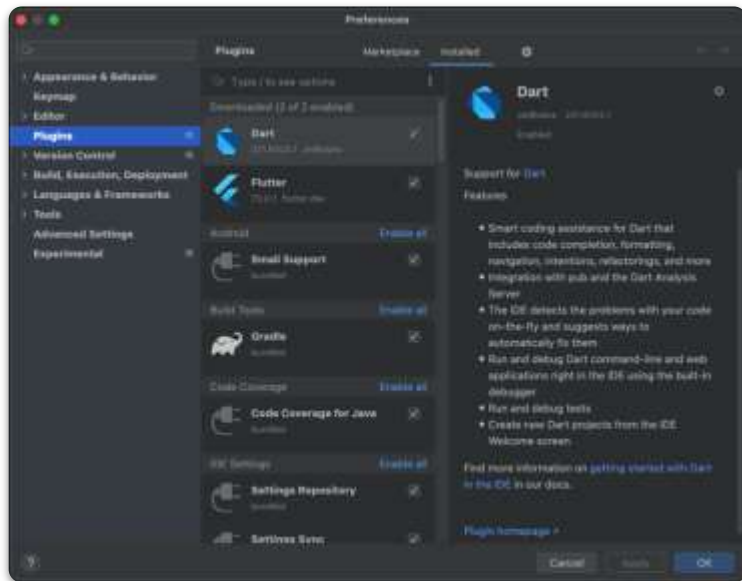


## IntelliJ IDEA

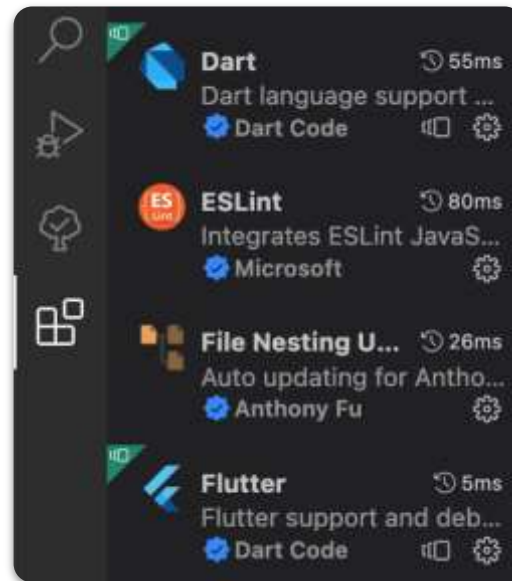
IDE très **professionnel** et  
**complet** mais **payant** !



# Installez les **extensions** pour vos IDE



Pour **Android Studio** et **IntelliJ**



Pour **VSCode**

# Développer pour **chaque plateforme**

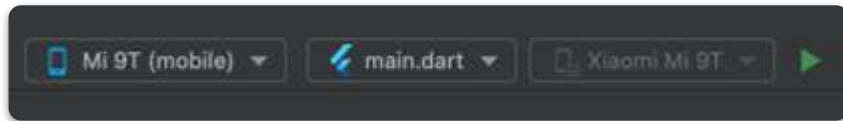
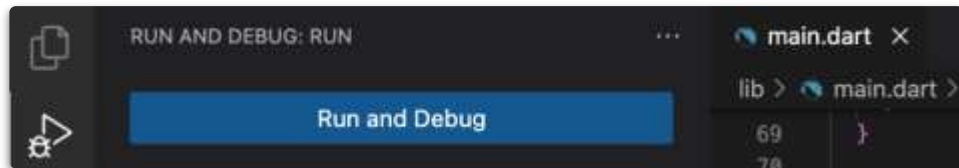


Le code Flutter peut être  
**écrit sur n'importe  
quelle plateforme** mais  
à des **contraintes de  
compilation**

| Plateforme         | Contraintes de compilation              |
|--------------------|-----------------------------------------|
| <b>Android</b>     | Android Studio (N'importe quel support) |
| <b>IOS / MacOS</b> | XCode (et donc disposer d'un mac)       |
| <b>Windows</b>     | Visual Studio sur Windows               |
| <b>Linux</b>       | Snapcraft sur Ubuntu                    |
| <b>Web</b>         | N'importe quel support                  |

# Créez, lancez et construisez votre projet

```
> flutter create nom_du_projet  
> cd nom_du_projet  
> flutter run
```



```
> flutter build apk
```

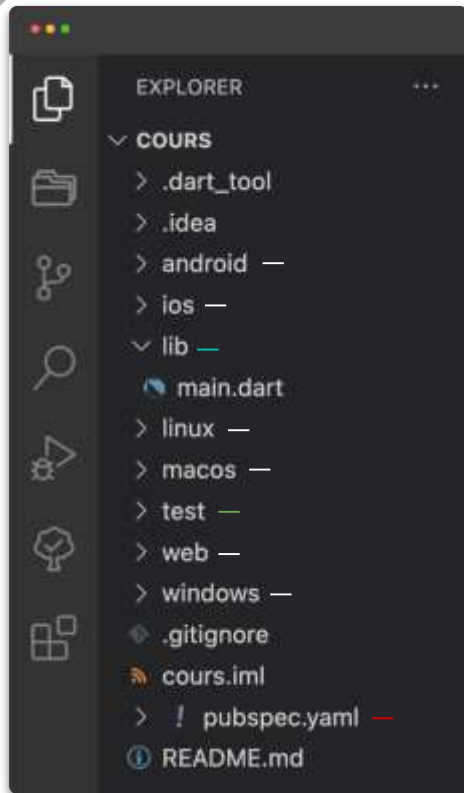
# Pas de téléphone ? Passez sur **émulateur** !



## **Petit conseil**

Privilégiez l'émulateur  
iphone sur MacOS pour de  
meilleures performances !

# L'architecture d'un projet



- ☐ Dossiers spécifiques à chaque plateforme
- ☒ Dossier pour la partie développement Flutter
- ☒ Dossier pour les tests
- ☒ Ensemble des fichiers (nesting pour ma part) de configuration du projet et de gestion des packages



04

# Que faut-il **apprendre ?**

Beaucoup de choses

# Tout est **widget** !



# Tout est **widget** !

```
void main() {  
  runApp(const MonApplication());  
}  
  
class MonApplication extends StatelessWidget {  
  const MonApplication({Key? key}) : super(key: key);  
  
  @override  
  Widget build(BuildContext context) {  
    return MaterialApp(  
      title: 'Mon premier widget',  
      home: Scaffold(  
        appBar: AppBar(  
          title: const Text('Mon premier widget'),  
        ),  
        body: const Center(  
          child: Text('Hello World'),  
        ),  
      ),  
    );  
  }  
}
```

**WEB**

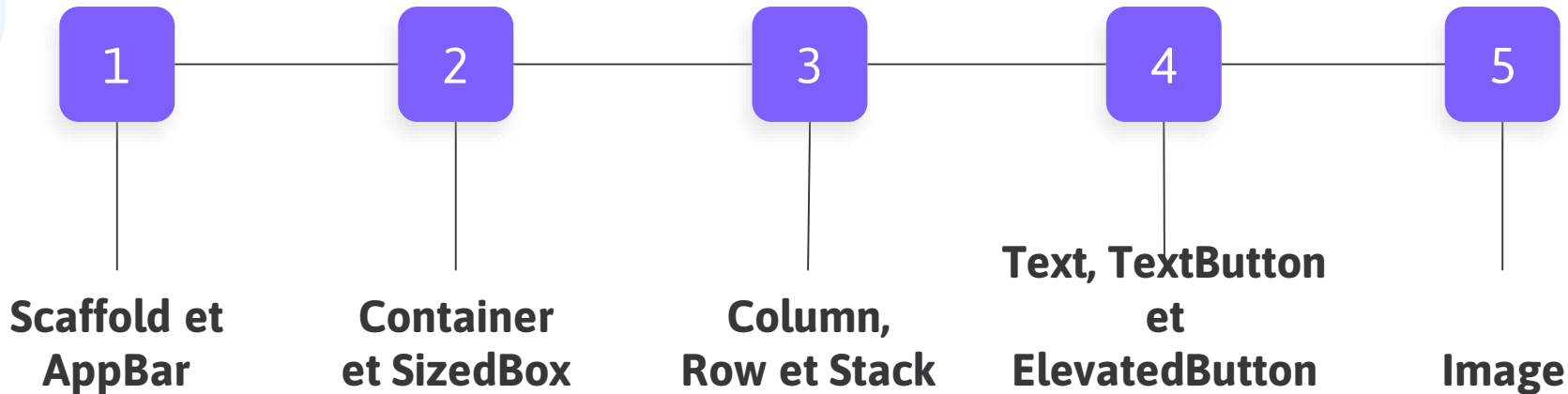




# Widget of the Week



# Les **principaux** Widgets



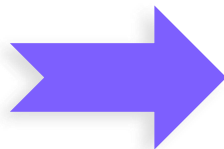
----

```
import 'package:flutter/material.dart';

void main() {
  runApp(const MyPage());
}

class MyPage extends StatelessWidget {
  const MyPage({Key? key}) : super(key: key);

  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      home: Scaffold(
        body: Column(
          children: [
            Container(
              color: Colors.blue,
              child: Text("C'est"),
            ),
            Text('Trop'),
            SizedBox(height: 20),
            Text('Facile ! :)'),
            Container(
              width: 20,
              height: 20,
              color: Colors.yellow,
            ),
            Container(
              color: Colors.green,
              child: Text('et un peu moche :/'),
            ),
            ElevatedButton(
              child: Text('Je suis un bouton avec null dans onPressed'),
              onPressed: null,
            ),
          ],
        ),
      ),
    );
  }
}
```



C'est

Trop

Facile ! :)

et un peu moche :/

Je suis un bouton avec null dans onPressed

# Les 2 grands types de Widgets

Son état ne peut plus être modifié une fois qu'il a été créé : il est immuable

**Stateless**

**Stateful**

Son état peut être modifié après sa création grâce au setState

# Stateless

```
class MonWidgetStateless extends StatelessWidget {  
  const MonWidgetStateless({Key? key}) : super(key: key);  
  
  @override  
  Widget build(BuildContext context) {  
    return Container(  
      color: Colors.red,  
      child: const Text('Hello World'),  
    );  
  }  
}
```

```
class MonWidgetStateful extends StatefulWidget {  
  const MonWidgetStateful({Key? key}) : super(key: key);  
  
  @override  
  State<MonWidgetStateful> createState() => _MonWidgetStatefulState();  
}  
  
class _MonWidgetStatefulState extends State<MonWidgetStateful> {  
  int counter = 0;  
  
  @override  
  Widget build(BuildContext context) {  
    return ElevatedButton(  
      onPressed: () {  
        setState(() => counter++);  
      },  
      child: Text('Increment : $counter'),  
    );  
  }  
}
```

# Stateful

# La gestion d'état

Il existe un **grand nombre de packages** pour la gestion d'état en plus du `stateful` (avec son `setState`) de bas niveau proposé nativement par Flutter.

**Provider**

P

R

**Redux** (comme pour react)

Quelques  
exemples

**Riverpod**

R

B

**BLoC**

# Un **récap** sur Flutter

